AI ACADEMY · LEVEL 2 · GRADE 7 · WEEK 35 · STUDENT LESSON

Improve and Challenge

Learn to understand, design, build, test and explain AI systems responsibly.

Project: Transparent Book Recommender

THE CONNECTED WEEK

Locate the cause. Repair one rule. Challenge the result.

Make a focused contract-based repair and preserve independent regression evidence.

Keep one habit: Boundary failure → precise comparison → independent expectations → integration checks → change record.

MAIN LESSON · SECTION 1

Did this repair fix the rule without hiding another failure?

> Preserve the contract, original failure and full regression evidence.

Your learning goals

I can repair a specific observed failure.

I can check that earlier behavior still works.

I can keep a truthful change and failure record.

Recall why old passing cases matter

- Close your notes first. A rough first answer helps us choose the right starting point; it is not a score for your ability.

- Why rerun old passing cases after a fix?

Your end goal

By the end, I can make a focused repair and challenge it with **regression** and order-invariance tests.

▼ Testing extension · Unit checks versus the full pipeline

Why can testing only valid_item miss a pipeline bug?

> **My readiness explanation**

▼ Earlier wording, alternatives and picture transcripts

Your learning target

Remember before reading

Without opening the old lesson, explain one key idea from Test Diverse Users. Give an example and a mistake that the idea helps you avoid.

Without opening the old lesson, explain one key idea from Project Problem and Success Test. Give an example and a mistake that the idea helps you avoid.

Without opening the old lesson, explain one key idea from **Evidence** and Sources. Give an example and a mistake that the idea helps you avoid.

After trying, check the relevant lesson or ask your teacher. Change the part of your explanation that the **evidence** shows was wrong.

Week 35: Improve and Challenge

A **controlled improvement** changes one design choice and reruns the same tests.



THINKER MISSION

Compare **before/after** scores and explain the **tradeoff**.

Thinker name and date:

> **Write or draw your reasoning and evidence.**

Readiness for the connected explanation

Readiness for Testing functions and interfaces

This week’s end goal

MAIN LESSON · SECTION 2

Find the comparison that excludes equality

> A focused repair changes the implementation to match the written requirement.

The at-most duration bug

A fictional chooser wrongly excludes an activity at the exact time limit because it uses minutes < limit. The requirement says at most, so equality must qualify.

Repair that condition, then test below, at and above the boundary plus earlier tie and invalid-**input** behavior.

▼ **Testing extension · Independent expectations**

Test the chooser at two scales. A **unit test** checks one validator or score calculation. An **integration test** runs validation through selection and reporting.

Write expected outcomes from the specification, not by calling the same function twice.

▼ **Earlier wording, alternatives and picture transcripts**

PICTURE STORY

Notice → Predict → Explain

Begin with visible **evidence**. Do not explain more than the picture and information support.



MYSTERY

The learners must use improve and challenge to solve a problem. What information do they have? What decision or calculation do they make? What would count as **evidence** that it worked?

Three **observation**s (what you can point to):

> **Write or draw your reasoning and evidence.**

My **prediction** and the clue supporting it:

> **Write or draw your reasoning and evidence.**

A closer explanation

Apply the connected idea: Testing functions and interfaces

MAIN LESSON · SECTION 3

Trace below, at and above the repaired boundary

> The predicate assumes validated durations; it does not validate raw data.

Predict and run the complete boundary script

```
def eligible(minutes, limit):
    return minutes <= limit

for minutes, expected in [(9, True), (10, True), (11, False)]:
    actual = eligible(minutes, 10)
    assert actual == expected
    print(minutes, actual)
```

Trace the old and revised comparison

Follow the evidence

STEP OR CASE	EVIDENCE	MEANING
Before fix	10 < 10 is False	Violates at most rule
After fix	10 <= 10 is True	Boundary accepted
Neighbor check	11 <= 10 is False	Over-limit still excluded

Read each row from left to right and connect the evidence to the stated rule.

The repaired predicate prints 9 True, 10 True and 11 False. It assumes validated numeric durations; it is not a replacement for the schema validator.

A focused change is easier to explain and evaluate than unrelated edits made at the same time.

At most includes equality. For a limit of 10, the durations 9 and 10 qualify while 11 does not. The strict comparison minutes < limit wrongly rejects 10; changing it to minutes <= limit repairs that boundary.

Check below, at and above the boundary, then rerun earlier tie and invalid-**input** cases. Keep the original failure record so another person can understand what changed and which **evidence** supports the repair.

Explain the deciding step

- Point to the **input**, the deciding step and the result in this week's visual. Explain the same step in ordinary words. A picture helps when you can say what each part represents.

▼ Testing extension · Complete score unit checks

```
def score(item):
    return 2 * item["fun"] + item["learning"]

cases = [( {"fun": 0, "learning": 0}, 0),
         ( {"fun": 3, "learning": 2}, 8),
         ( {"fun": 5, "learning": 5}, 15)]
for item, expected in cases:
    assert score(item) == expected
print("3 score cases passed")
```

Follow the evidence

STEP OR CASE	EVIDENCE	MEANING
Unit	Score 3 and 2	Expect 8
Integration	Invalid high-score row	Must never win
Report	No eligible rows	Explain no suggestion

Read each row from left to right and connect the evidence to the stated rule.

The code prints its success message only if all assertions pass. It checks the scoring component but says nothing about duration filtering, malformed records, ties or reporting. Add focused cases for those requirements.

Use the production functions from earlier weeks rather than rewriting a second copy just for tests.

A score **unit test** asks whether the scoring function calculates the specified number. An **integration test** asks whether validation, filtering, scoring and reporting work together.

A correct score does not establish that invalid records were rejected or that a tie was resolved correctly. Predict the full report from the **input** records before running.

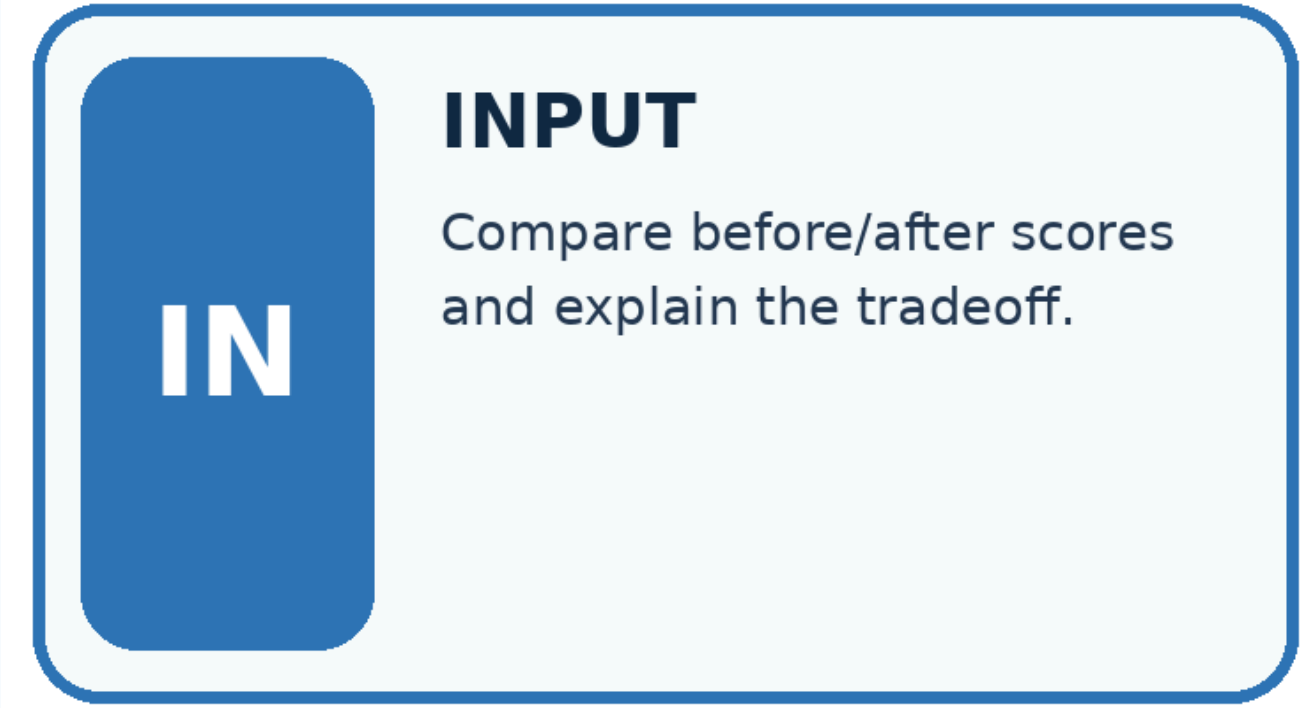
Check its counts and reasons as well as its selected name, and repeat a tie case with reversed **input** order.

▼ Earlier wording, alternatives and picture transcripts

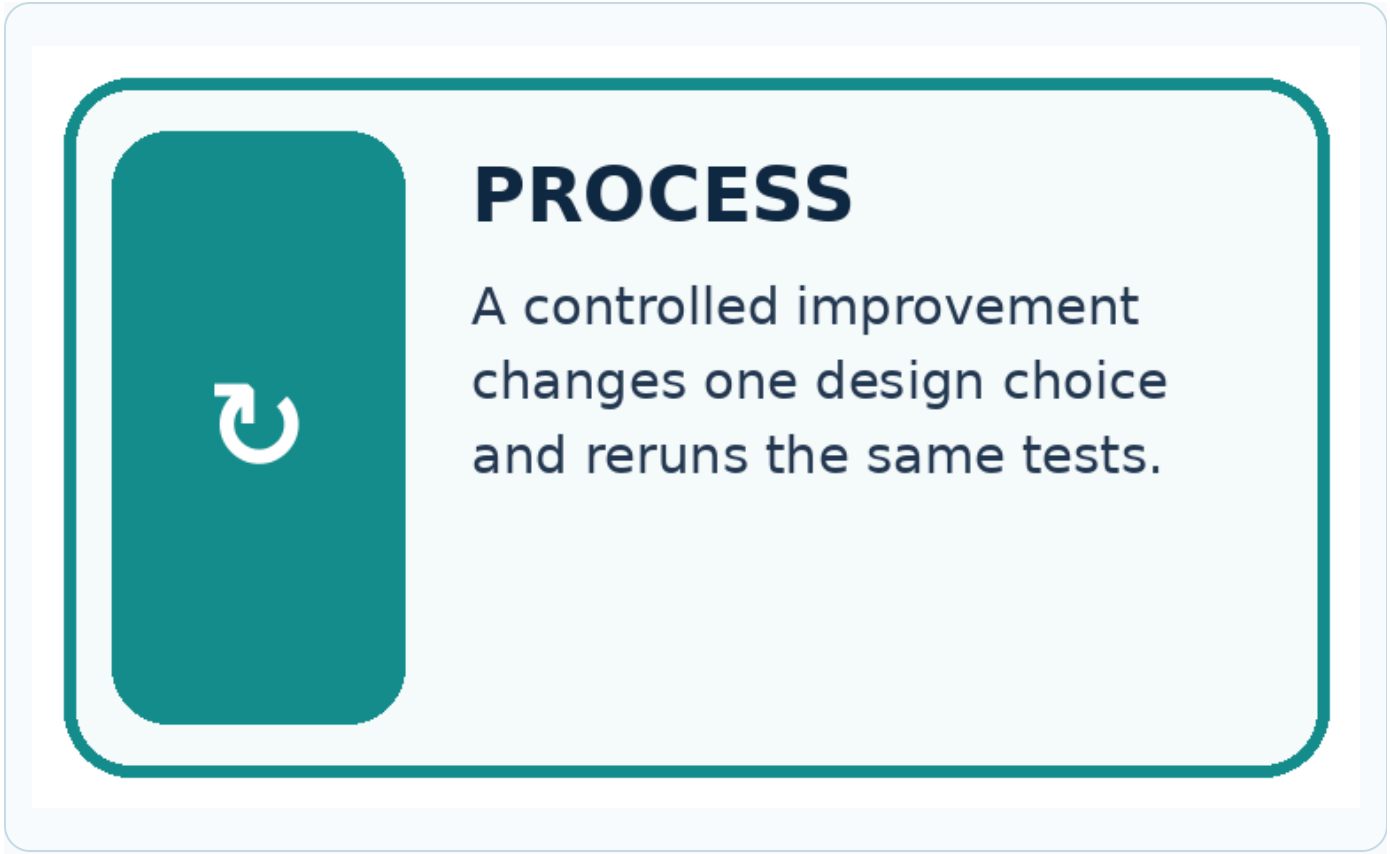
VISUAL **MECHANISM**

Input → **Process** → **Output**

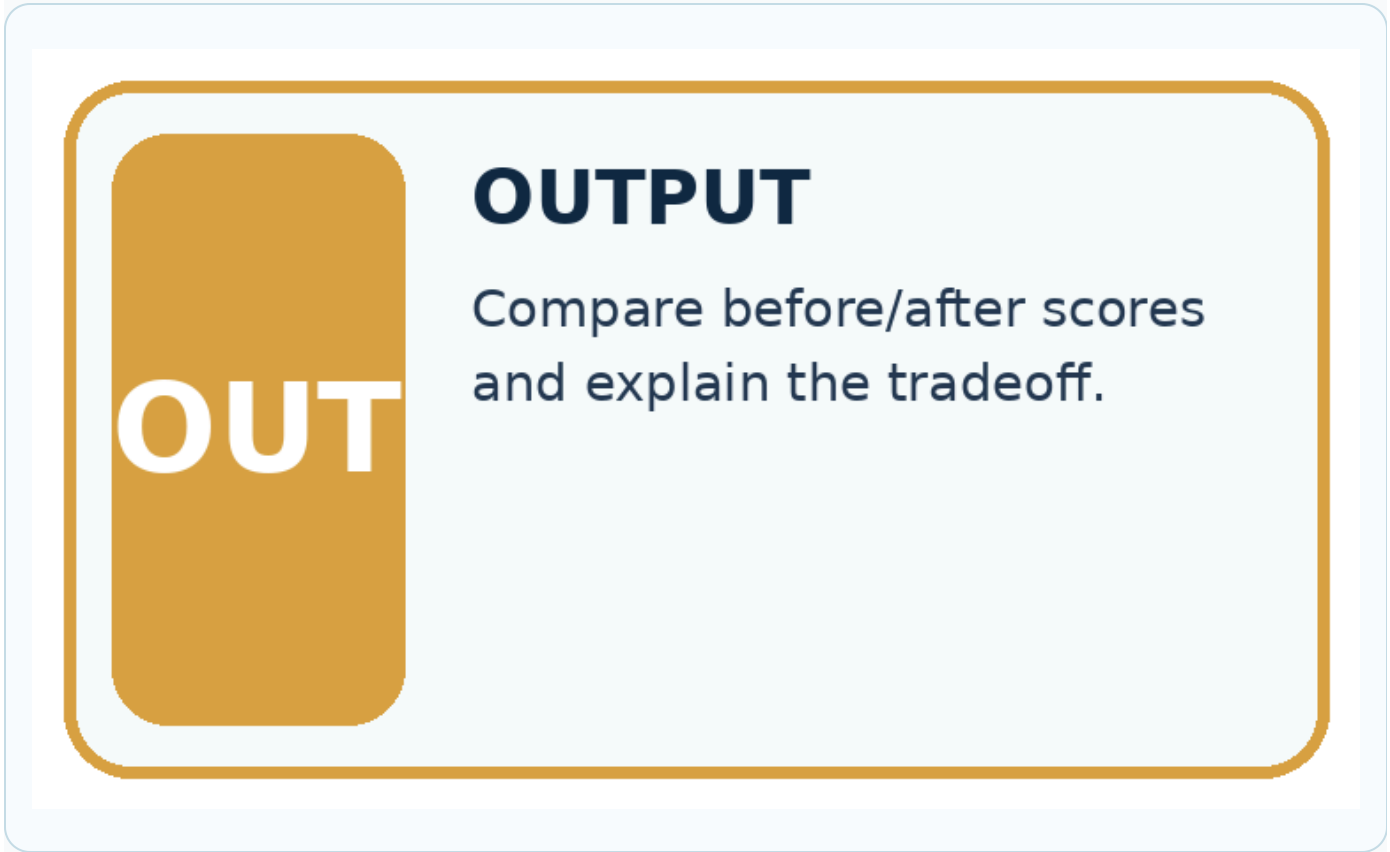
Trace the information. At each stage, ask what changed and how you could test it.



↓


↻
PROCESS
A controlled improvement changes one design choice and reruns the same tests.

↓


OUT
OUTPUT
Compare before/after scores and explain the tradeoff.

Which stage could fail even when the other two are correct? Explain:

> **Write or draw your reasoning and evidence.**

A closer explanation

Repair the rule, then keep the evidence

Apply the connected idea: Testing functions and interfaces

Test the connections as well as the pieces

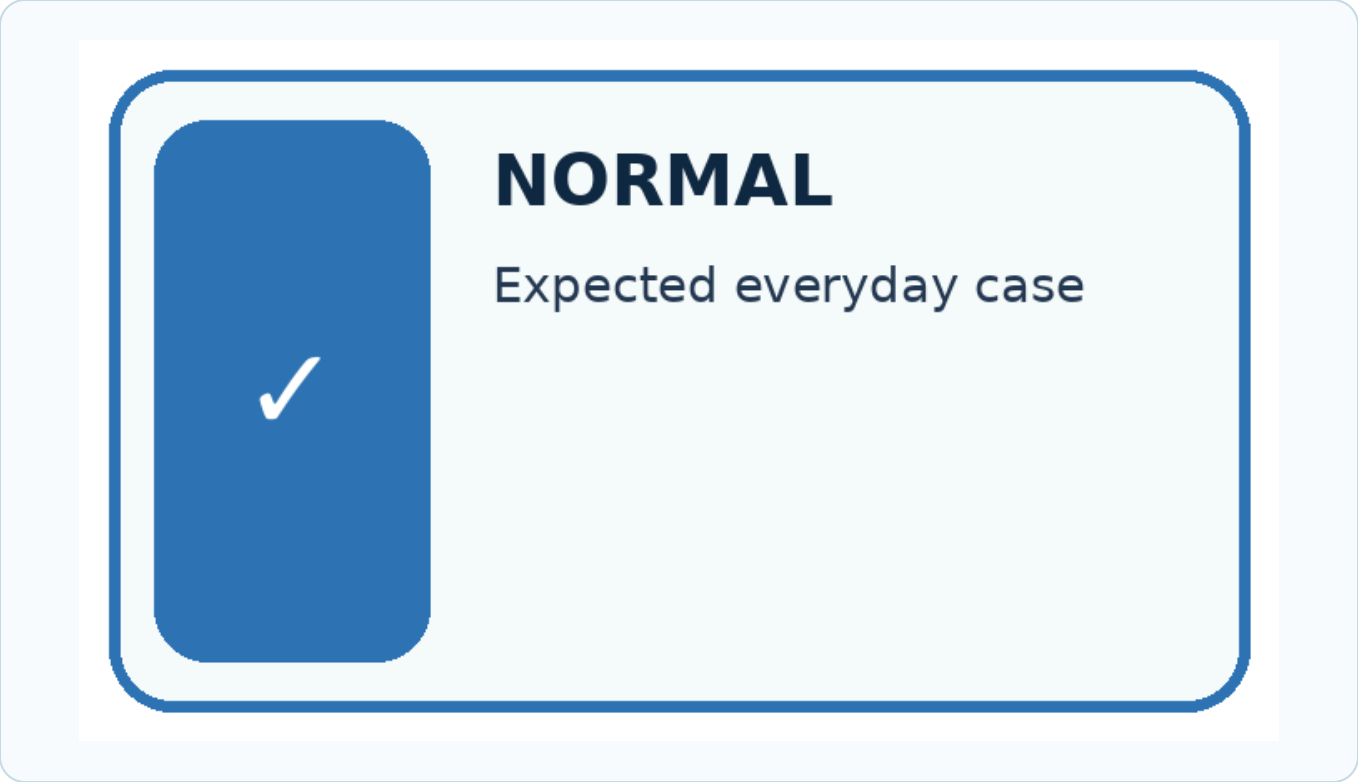
Connect the picture and the explanation

MAIN LESSON · SECTION 4

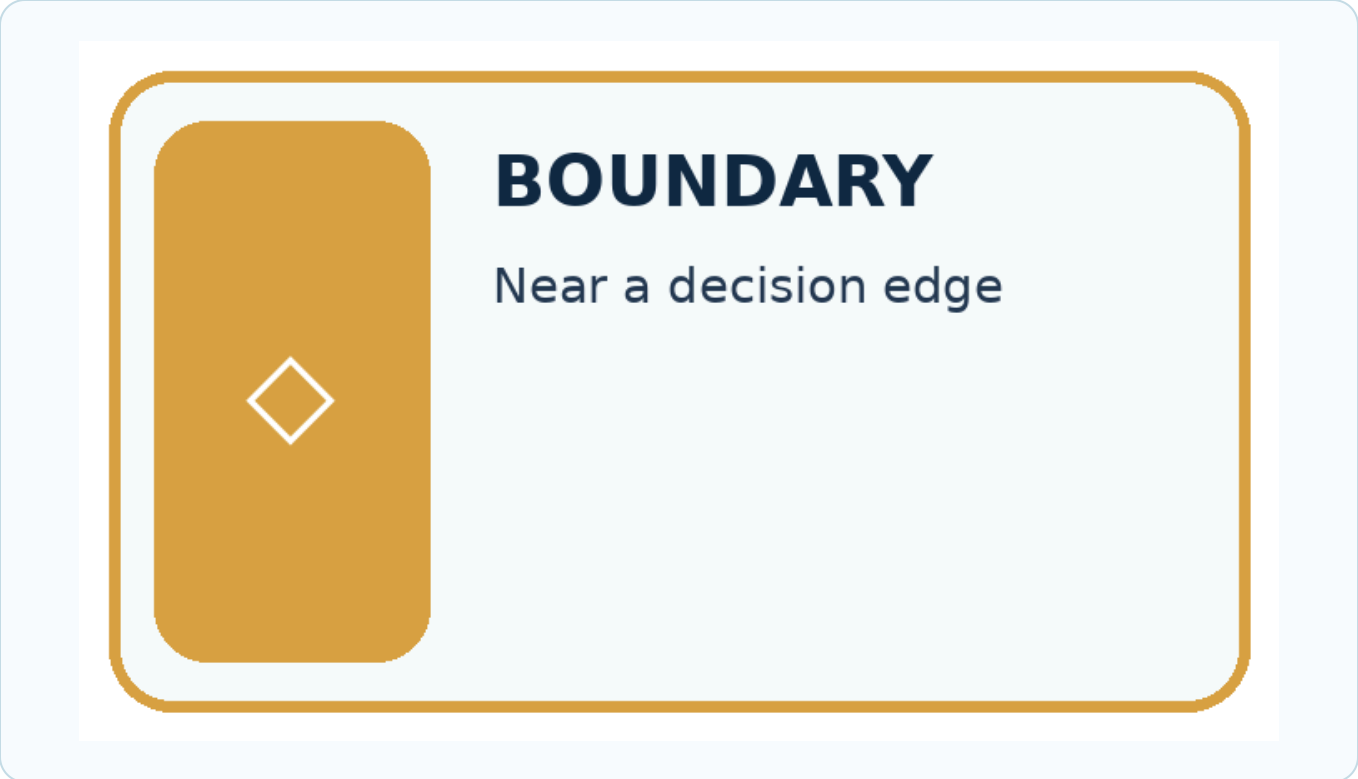
Compare a true repair with a changed requirement

> Changing the expected answer cannot repair a contract violation.

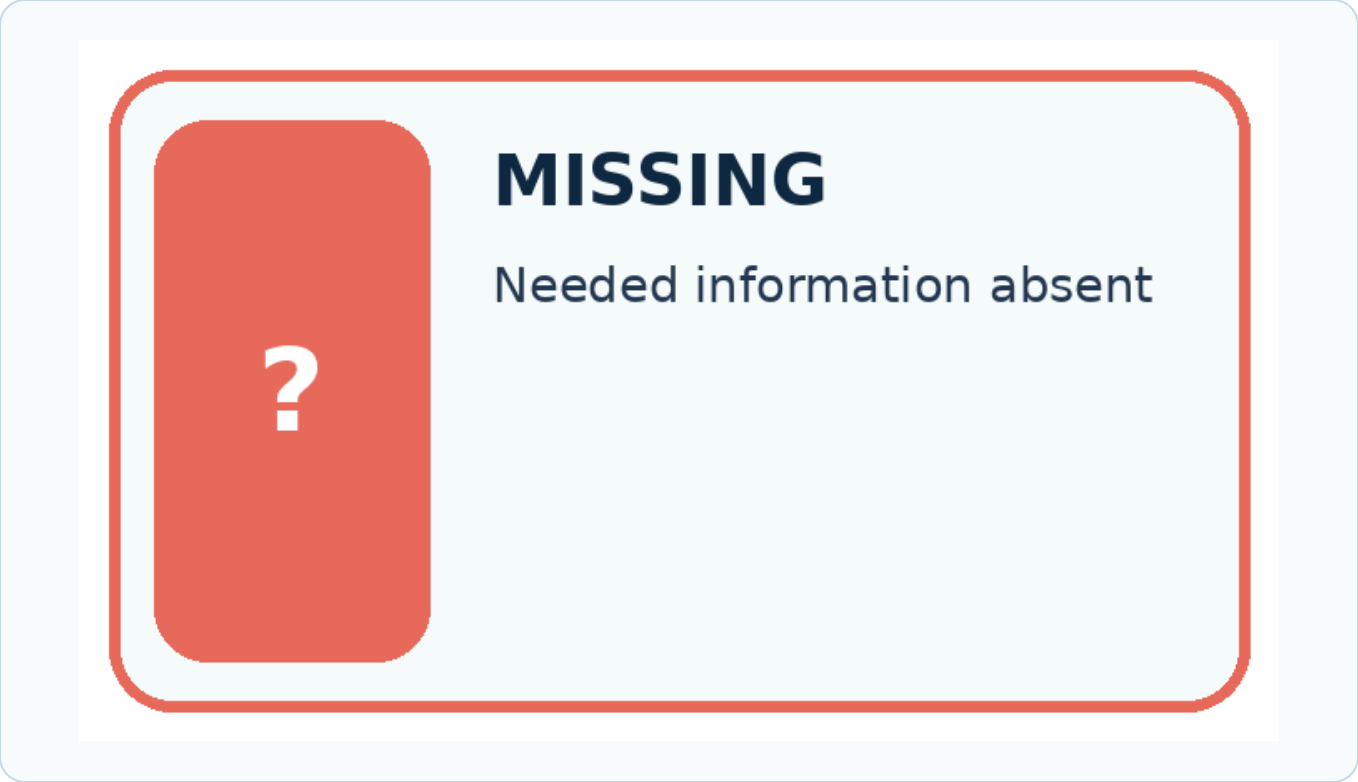
Four pictures with a purpose



Normal case: predict the result and deciding evidence.



Boundary case: identify the exact limit.



Missing information: name what must be clarified.



Transfer case: explain which procedure still applies.

Choose a revealing case

- Which case is most likely to reveal a hidden assumption? Why?

> **Write or draw your reasoning and evidence.**

Predict the supplied changed case

- Change: An alternative patch uses pages <= 101 instead of the required pages <= 100.

▼ Check after predicting

Prediction: C is wrongly admitted as well as A.

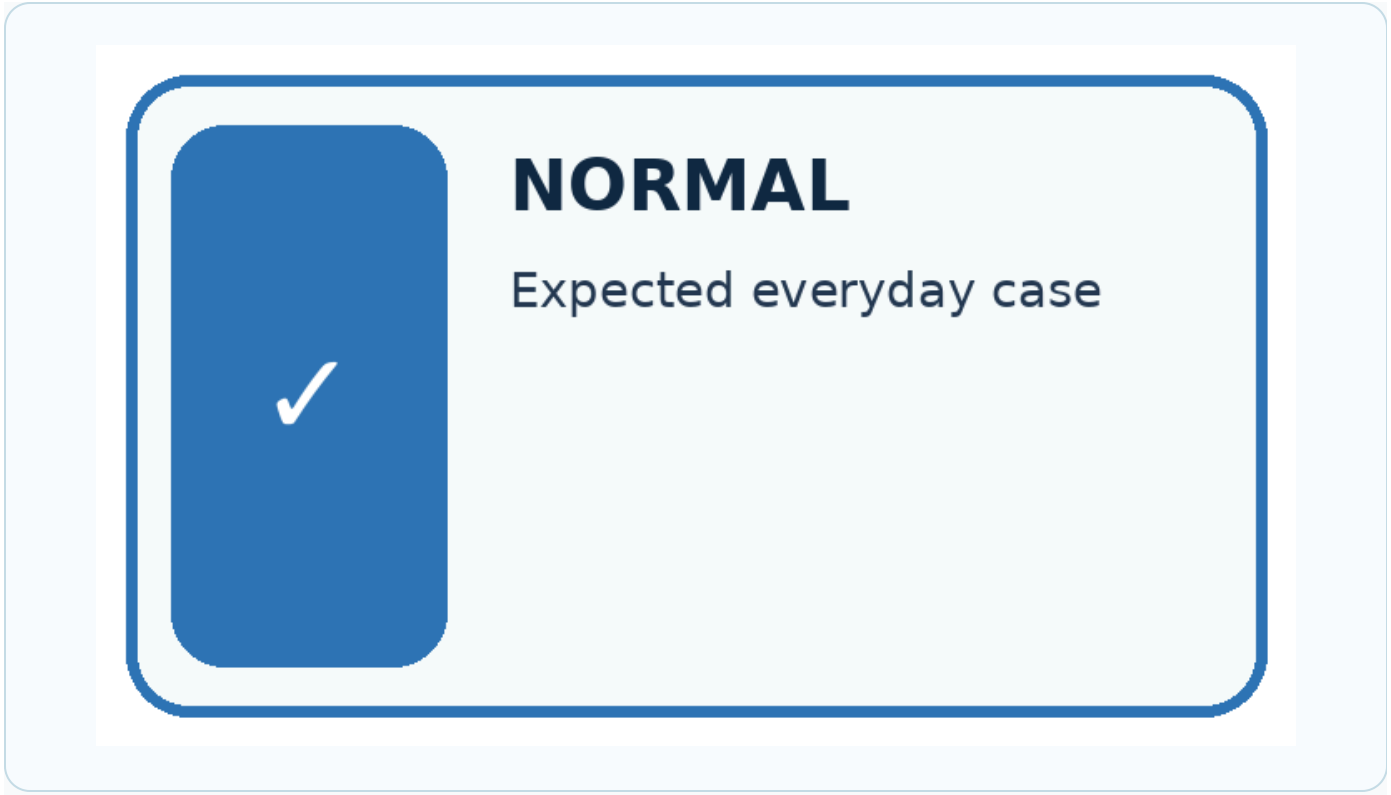
Reason: Changing the maximum alters the contract instead of repairing its comparison.

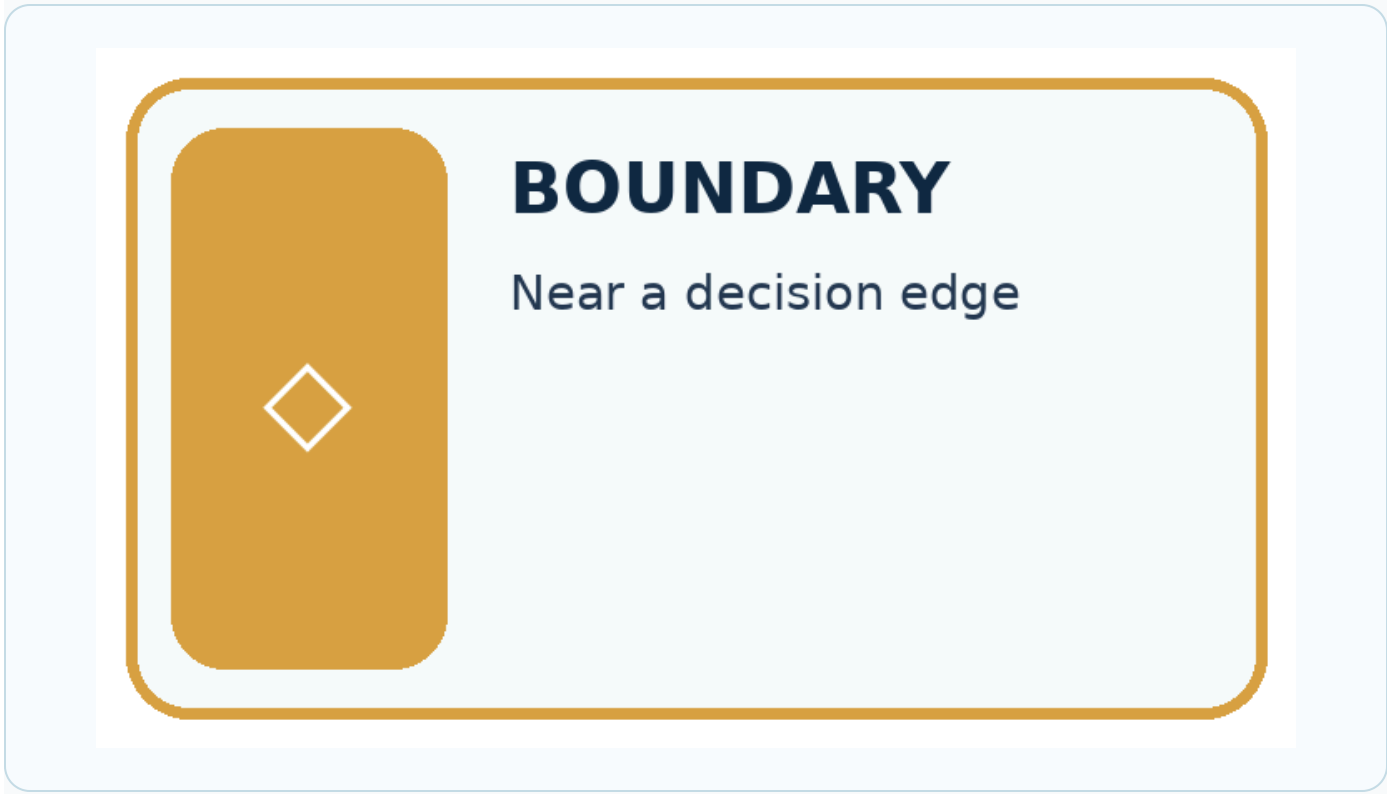
▼ Earlier wording, alternatives and picture transcripts

CASE GALLERY

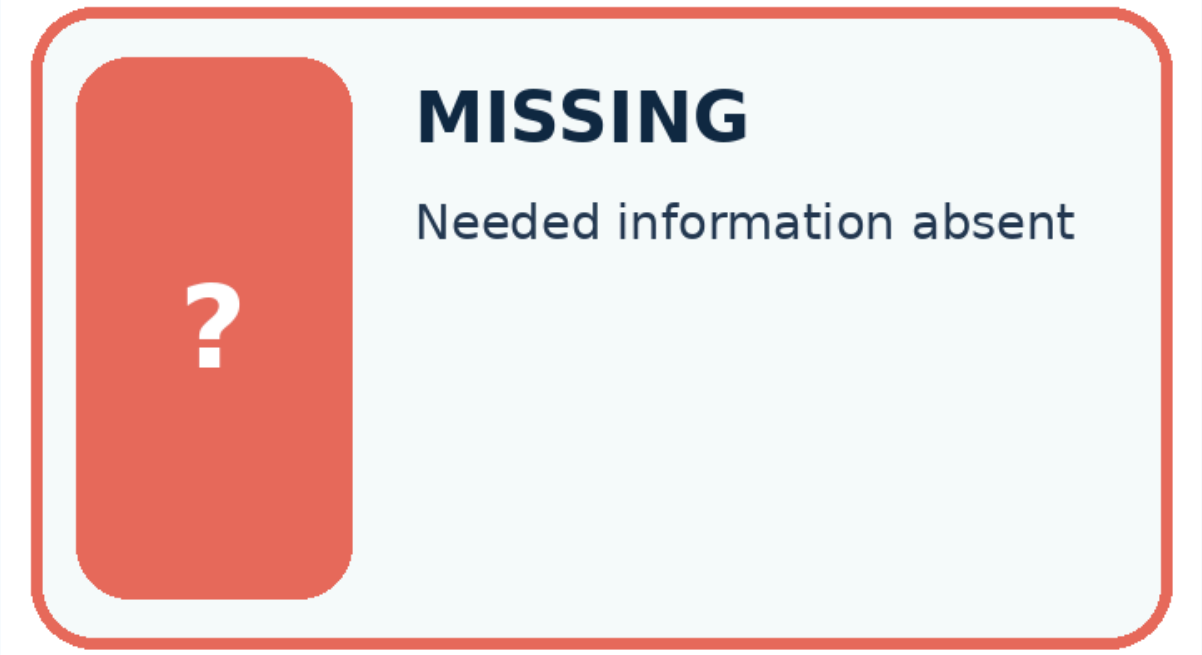
Four Tests, Four Kinds of **Evidence**

A system is not understood until it survives more than one comfortable example.





NORMAL CASE: predict and name **evidence**. BOUNDARY CASE: predict and name **evidence**.





MISSING INFORMATION: predict and name **evidence**. TRANSFER CASE: predict and name **evidence**.

Change one thing in the pictured case

MAIN LESSON · SECTION 6

Challenge the duration fix, then inspect a separate page-limit bug

> Keep the10-minute predicate and100-page ranking as separate examples.

Predict the limit-plus-one counterexample

Would changing the condition to minutes <= limit + 1 correctly repair the boundary? Explain with a revealing input.

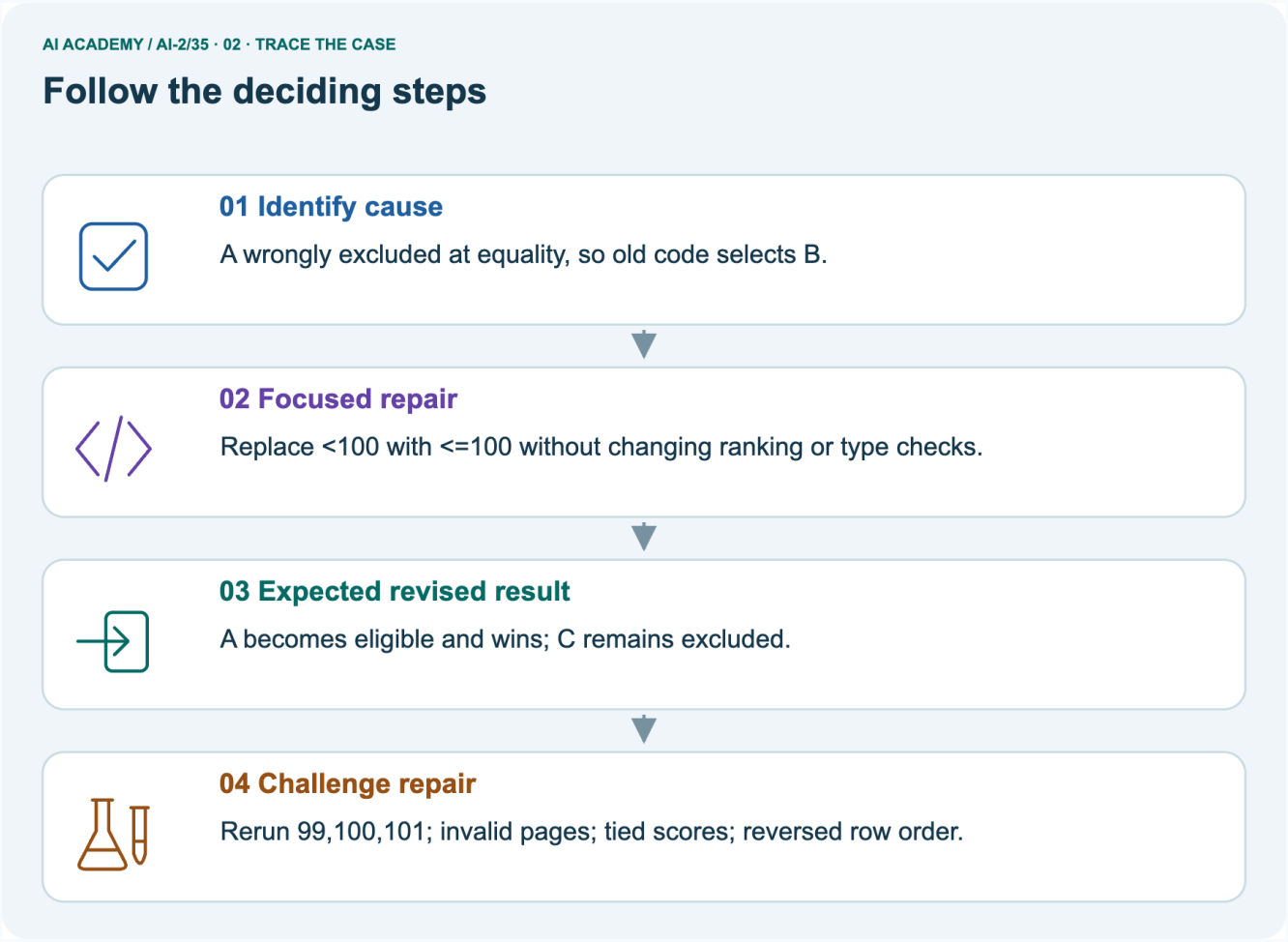
Read the separate100-page contract and rows

A fictional recommender excludes books at a page limit that is intended to include equality.

Contract: pages<=100; highest interest; ties choose smaller ID. Old code uses pages<100.

Rows A pages100 interest5; B pages90 interest3; C pages101 interest5.

Trace the first faulty comparison and focused repair



Follow the labelled picture with the matching explanation. Open the vector diagram

Read the picture one step at a time

STEP	WHAT HAPPENS AND WHY
1. Identify cause	A wrongly excluded at equality, so old code selects B.
2. Focused repair	Replace <100 with <=100 without changing ranking or type checks.
3. Expected revised result	A becomes eligible and wins; C remains excluded.
4. Challenge repair	Rerun 99,100,101; invalid pages; tied scores; reversed row order.

Match each pictured step to the supplied facts. This is a worked teaching case, not an independent assessment.

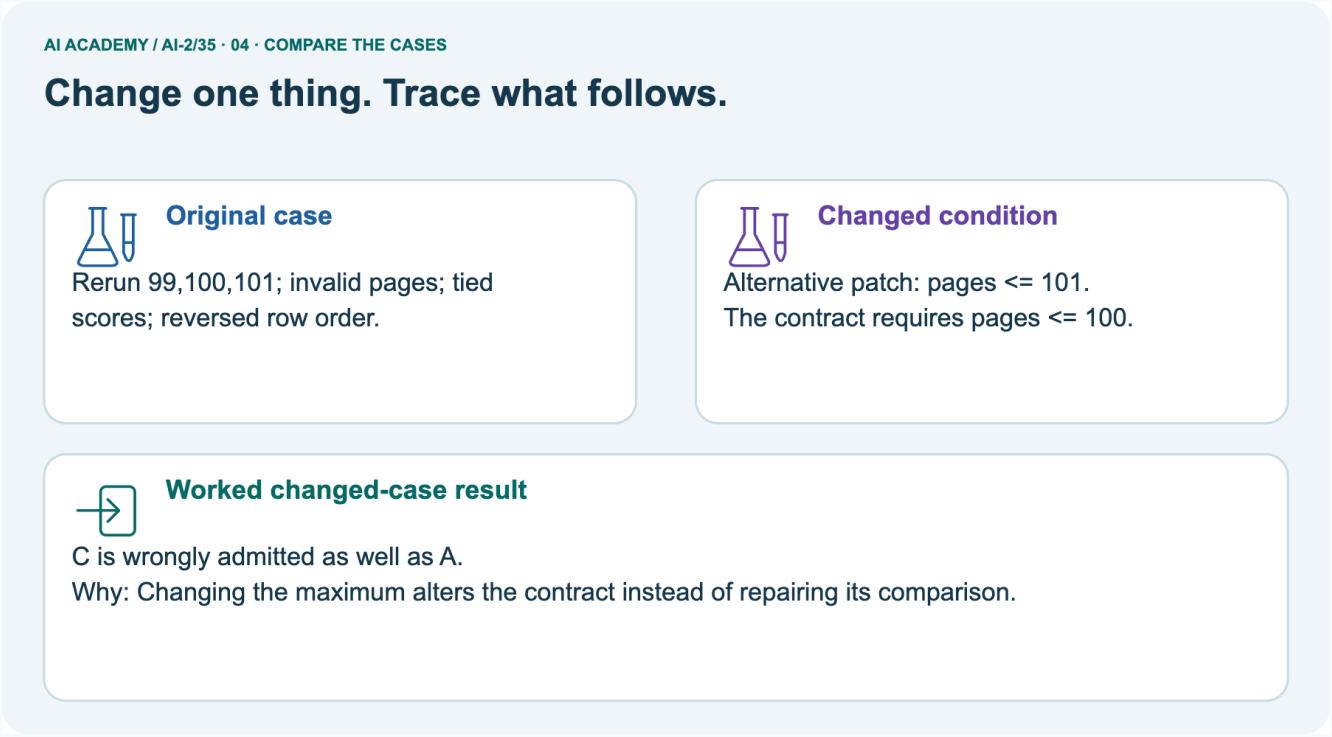
A focused change makes the claim about what caused the error testable: the equality comparison rule caused this specific failure. **Regression** tests check surrounding behavior that should remain unchanged.

A tie case and reversed **input** order are necessary because a correct boundary does not ensure deterministic selection. Log the old failure and revised replay separately.

Predict the explicit pages<=101 alternative

- Change one thing: An alternative patch uses pages <= 101 instead of the required pages <= 100. Predict the result before checking the explanation. What remains the same?

▼ Check after predicting



Follow the labelled picture with the matching explanation. Open the vector diagram

Explain why both equality and excess need tests

- Passing a small **regression suite** does not prove all behavior unchanged; record the suite's actual coverage.

- Read the labelled picture: Why are tests at both 100 and 101 needed?

▼ Testing extension · A function cannot supply its own independent oracle

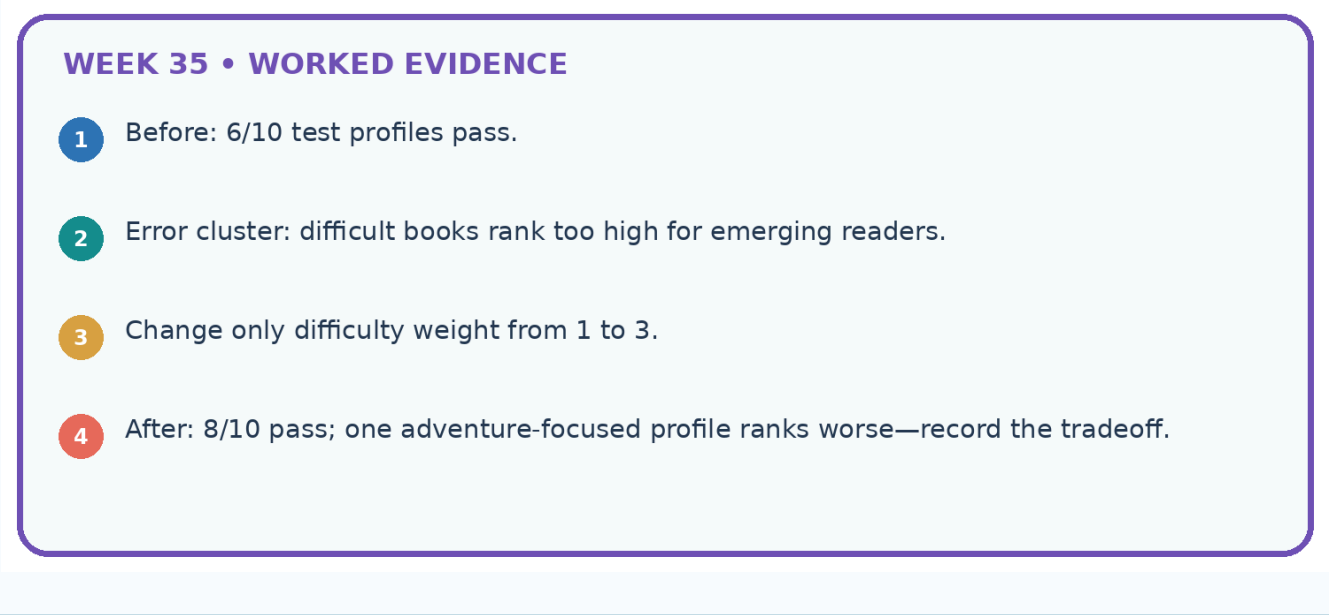
> **A test computes expected = score(item) then compares score(item) to expected. Why is it weak?**

▼ Earlier wording, alternatives and picture transcripts

WORKED **EVIDENCE**

Reason Step by Step

Predict first. Then check each step. A correct answer without visible reasoning is incomplete **evidence**.



1. Before: 6/10 test profiles pass.

2. Error cluster: difficult books rank too high for emerging readers.

3. Change only difficulty weight from 1 to 3.

4. After: 8/10 pass; one adventure-focused profile ranks worse—record the **tradeoff**.

Recalculate, retrace, or restate the reasoning in your own representation:

> **Write or draw your reasoning and evidence.**

CHECK Did the **evidence** answer the exact question? Did the total, path, or source record stay consistent?

A closer explanation

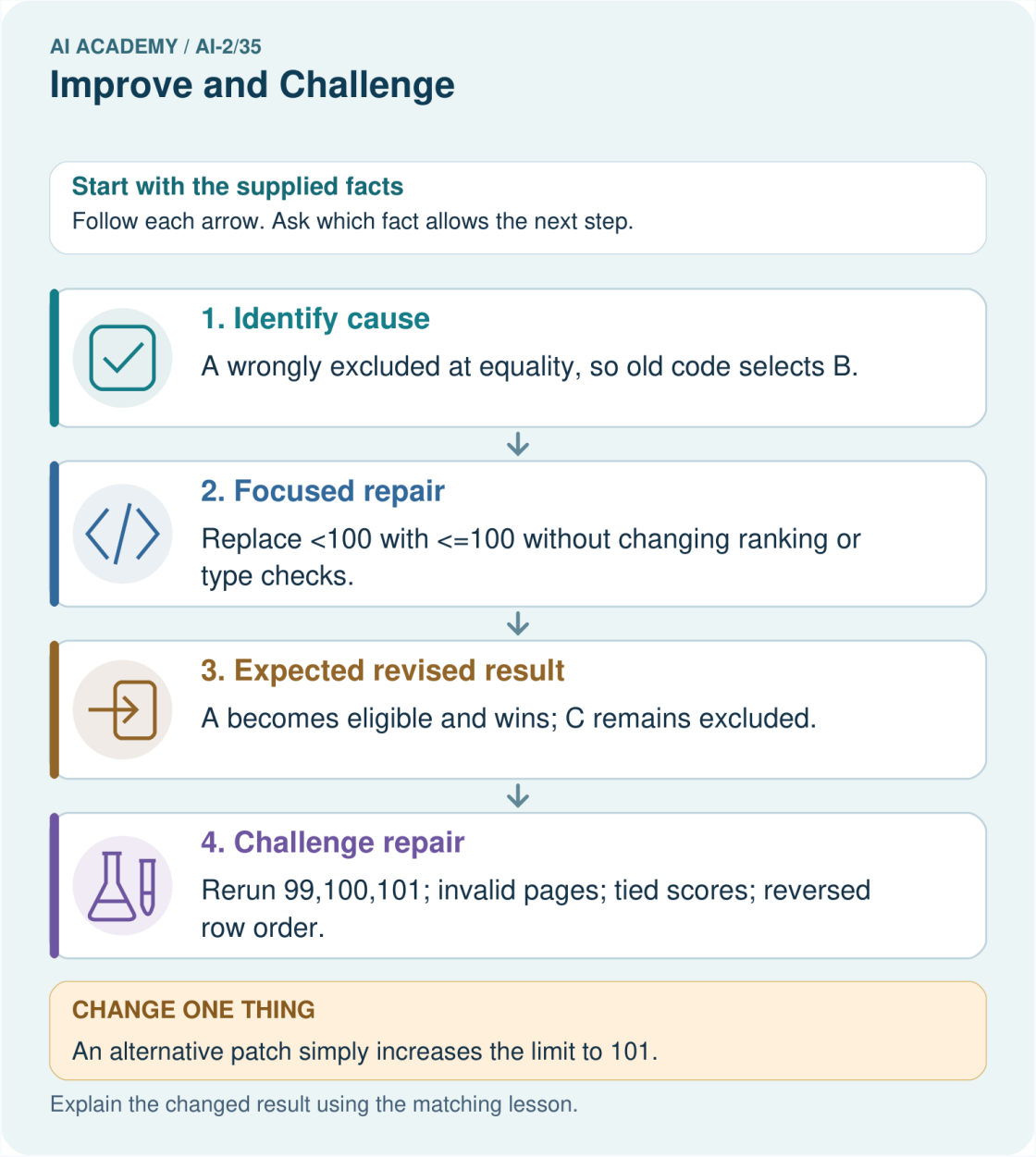
Apply the connected idea: Testing functions and interfaces

Learn from the worked case

Cover the result before checking it. Say what is given, choose the procedure, and follow one step at a time. Then uncover the result and explain your first mismatch. Ask why a step is allowed, not only what number it produces.

A pictorial worked case: Improve and Challenge

The facts we will use



Follow the labelled picture with the matching explanation. Open the vector diagram

Why this result follows

What this example does not establish

MAIN LESSON · SECTION 7

Repair the rule instead of moving the goalposts

> Keep failed inputs and independently derived expectations.

Find the unsupported change

Claim to repair: The old boundary test expected True but returned False, so change the expected value to False. The contract says at most; after validation equality must qualify. Preserve that independent expectation and repair the strict comparison. If the policy itself really changes, version the new policy and tests explicitly rather than presenting it as the same repair.

Propose a test that challenges the repair

Record the bug, change, evidence and one remaining limit without claiming all bugs are gone.

▼ Earlier wording, alternatives and picture transcripts

ERROR DETECTIVE

Compare → Diagnose → Repair

Do not patch the answer. Find the earliest unsupported assumption, wrong step, or weak **evidence**.

!

COMPARE • DIAGNOSE

Weak idea: If the total score improves, no old behavior needs retesting. Strong move: locate evidence, change one cause, and retest.

Evidence record	Expected	Actual	Likely cause / next test
normal case	works		
boundary case	stated rule		
missing input	safe response		
fresh transfer	generalizes		

Repair one cause. Why is this repair better than changing several things?

> **Write or draw your reasoning and evidence.**

A closer explanation

A clean **regression** run does not erase the earlier failure. Keep **before/after evidence** and remaining limitations. If a fix was informed by reserved evaluation cases, report that contamination and obtain new independent cases.

Apply the connected idea: Testing functions and interfaces

Test results establish only the executed cases on the tested version. Retain failures and version information. Use explicit expected rejection behavior for invalid data instead of assuming every exception is acceptable.

> **What expected result should an empty-input integration case specify?**

MAIN LESSON · SECTION 8

Practise neighboring boundaries with less help

> An unchanged score unit check cannot establish that the pipeline still works.

Preserve the requirement and original failure

Changing the expected boundary result to False would make the test pass while violating the requirement. Keep the requirement and independently calculated expectation; repair the implementation.

If requirements truly change, document that as a policy change.

Complete main W2

Practise with less help: Test neighbors

Use workbook W2 for the connected example “Improving without hiding failures”. First fill the **input** and rule boxes. Try the middle steps before asking for a hint. After a hint, try the next step yourself.

Plan	Do	Check
What is given? Which rule or procedure applies?	Record each intermediate step.	Does the result follow? What remains unknown?

Repair an overclaim before discussion

- Use the worked case for Improving without hiding failures. Proposed explanation: “A successful run proves the conclusion”. Decide whether this explanation is supported.

  Point to the step or supplied fact that decides; explain your repair if needed.

- Answer on your own before discussion. If you are unsure, say which step you need help with.

Check your reasoning

Explain the separate pictured boundary

- Why are tests at both 100 and 101 needed?

Explain the picture

> **My explanation using a pictured step and the given facts**

Keep independent work separate

This is supported practice. Keep the separately named independent case for an attempt before feedback.

▼ Testing extension · Check full report fields

A test suite that only checks the winner name can miss wrong rejection counts. Check the full expected report: supplied, accepted, rejected, eligible, selected and reason.

Avoid relying on dictionary display order when the contract concerns values.

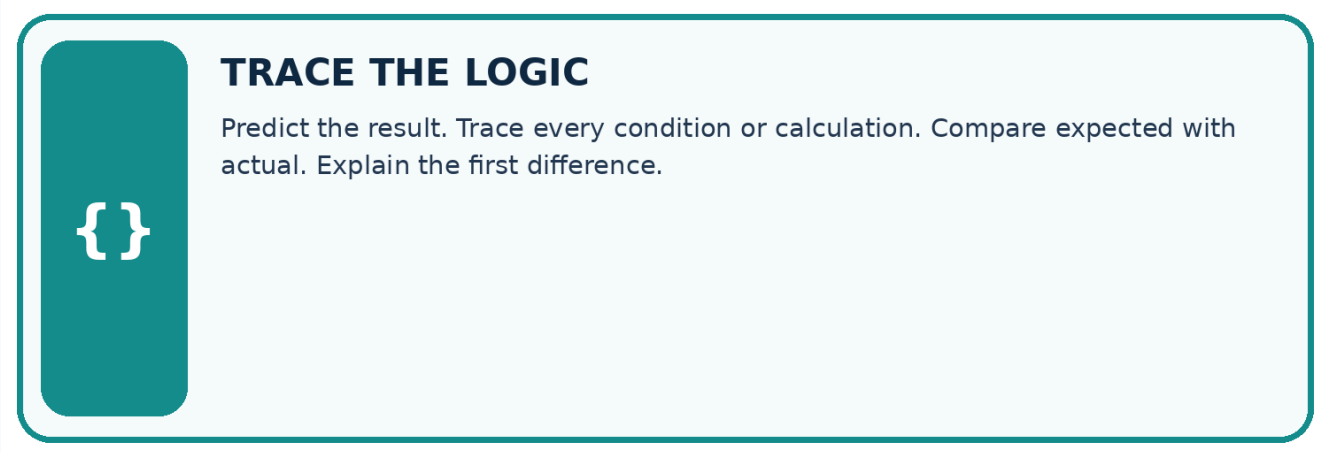
Use its named workbook W2. Identify the given **input**s, try the middle steps with less help, then explain your trace. Check this claim before group discussion: A successful run proves the conclusion

▼ Earlier wording, alternatives and picture transcripts

LOGIC STUDIO

Trace Before You Run

Block code, pseudocode, tables, and diagrams are different ways to expose the same reasoning.



SET **evidence**_record PREDICT expected_result RUN one normal case and one boundary case COMPARE actual_result with expected_result IF different: locate first cause; change one factor; RETEST EXPLAIN result, limitation, and next safe step

Trace one **input**. Which line changes the state or chooses a path?

Write or draw your reasoning and evidence.

A closer explanation

Apply the connected idea: Testing functions and interfaces

Explain your trace to a partner. Your partner points to one step to check. Swap roles on the next case. Each person writes their own explanation.

Practise the connected topic: Testing functions and interfaces

---

MAIN LESSON · SECTION 9

Run independent boundary and integration checks

> Keep demonstrated results separate from a historical repair claim.

Predict the fixture and evidence scope

First predict the existing standalone boundary script at9,10,11 for limit10 and explain why the old strict comparison differs at10. Then run the complete integration program below. It includes the exact Week33 implementation and its earlier demonstration prints, followed by five separately specified integration checks. Expected fixture: supplied3,accepted2,rejected1,eligible2; A/Bscore8, winnerA by name.

Reversal changes rejection row number but preserves the result and counts. The final success message appears only if every assertion passes.

Complete pipeline and assertions

```
from collections import Counter

def pipeline(raw, limit):
    if type(limit) is not int or not 0 <= limit <= 60:
        return {'error': 'invalid request'}
    names = Counter(r['name'] for r in raw if type(r) is dict
                    and type(r.get('name')) is str and r['name'])
    accepted, rejected = [], []
    for index, row in enumerate(raw, 1):
        reason = None
        if type(row) is not dict:
            reason = 'not a record'
        elif type(row.get('name')) is not str or not row['name']:
            reason = 'invalid name'
        elif names[row['name']] > 1:
            reason = 'duplicate name'
        else:
            for key, maximum in [('minutes',60),('fun',5),('learning',5)]:
                value = row.get(key)
                if type(value) is not int or not 0 <= value <= maximum:
                    reason = 'invalid ' + key
                    break
        if reason:
            rejected.append((index, reason))
        else:
            accepted.append(dict(row))
    eligible = [r for r in accepted if r['minutes'] <= limit]
    ranked = sorted(eligible, key=lambda r: (-(2*r['fun']+r['learning']),r['name']))
    scores = [(r['name'],2*r['fun']+r['learning']) for r in ranked]
    return {'winner': scores[0][0] if scores else None, 'ranked': scores,
            'accepted': len(accepted), 'rejected': rejected,
            'ineligible': [r['name'] for r in accepted if r['minutes']>limit]}

raw = [dict(name='A',minutes=20,fun=3,learning=2),
       dict(name='B',minutes=21,fun=5,learning=5),
       dict(name='C',minutes=12,fun=2,learning=4),
       dict(name='D',minutes=10,fun='4',learning=2)]
print(pipeline(raw,20))
print(pipeline(list(reversed(raw)),20)['ranked'])
print(pipeline(raw,0)['winner'])
print(pipeline(raw,True))
# Independent expectations for a separate integration fixture.
fixture = [dict(name='A',minutes=10,fun=2,learning=4),
           dict(name='B',minutes=10,fun=3,learning=2),
           dict(name='C',minutes=5,fun=9,learning=1)]
```

```
expected = {'winner':'A', 'ranked':[('A',8),('B',8)], 'accepted':2,
            'rejected':[(3,'invalid fun')], 'ineligible':[]}
assert pipeline(fixture,10) == expected
reverse = pipeline(list(reversed(fixture)),10)
assert reverse['winner']=='A' and reverse['ranked']==[('A',8),('B',8)]
assert reverse['accepted']==2 and reverse['rejected']==[(1,'invalid fun')]
assert pipeline([],10)=={'winner':None,'ranked':[],'accepted':0,
                         'rejected':[],'ineligible':[]}
assert pipeline(fixture,9)['winner'] is None
assert pipeline(fixture,True)=={'error':'invalid request'}
print('5 integration checks passed')
```

Challenge one comparison and preserve the record

1. Keep a change record: old condition, failing input, expected result from the contract, actual old result, precise repair, regression cases and remaining limits. The assertions check the repaired baseline; they do not replay a full historical faulty pipeline.

   To challenge the boundary in this local copy only, change the eligible-list comparison <=limit to <limit and predict which integration assertion fails; restore the contract before rerunning. Do not change the test expectation to follow the mutant. In the separate page example, the incorrect <=101 patch admits C101; this is an explicit different comparison, not the old strict <101.

   No10-profile before/after outputs are supplied, so do not claim a measured6/10→8/10 improvement from this investigation.

Keep prediction, result and first repair

Test / input · Prediction · Actual · Evidence / calculation · Change or keep?

> **Test / input:** normal
>
> **Prediction:** _____
>
> **Actual:** _____
>
> **Evidence / calculation:** _____
>
> **Change or keep?:** _____

Test / input: boundary

Prediction: _____

Actual: _____

Evidence / calculation: _____

Change or keep?: _____

Test / input: missing

Prediction: _____

Actual: _____

Evidence / calculation: _____

Change or keep?: _____

Test / input: fresh transfer

Prediction: _____

Actual: _____

Evidence / calculation: _____

Change or keep?: _____

What did the hardest test teach you that the normal test did not?

Write or draw your reasoning and evidence.

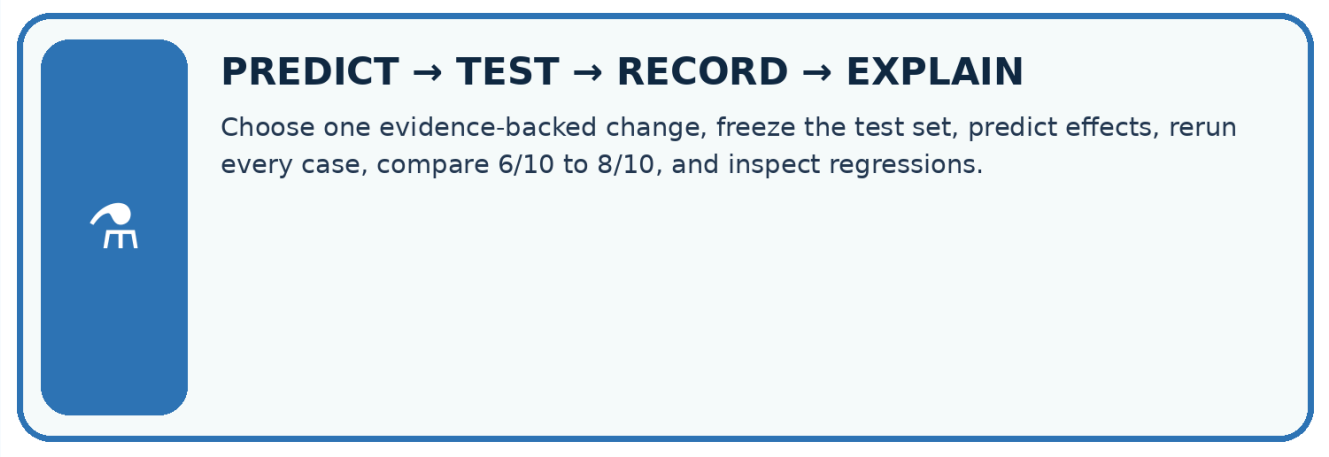
Before trying: what do I expect, and why? While trying: which step could first go wrong? Afterwards: what did I actually observe, and what should change? Keep “not run” if you only traced the procedure.

▼ Earlier wording, alternatives and picture transcripts

EVIDENCE LAB

Predict → Test → Record → Explain

Keep expected and actual results separate. Surprises are useful data when recorded honestly.



Run the supplied boundary script and exact Week33 pipeline with independently specified integration checks. Preserve any failed assertion, test one local comparison mutation, restore the requirement and rerun. The retained6/10→8/10 scenario is separate illustrative data, not an output of this lab.

Plan → try → check

MAIN LESSON · SECTION 14

Try alone: repair the fresh12-minute boundary

> Keep the contract and add concrete regression cases.

Independent brief and scope

Write the eligibility predicate for at most **12 minutes** and a **regression** set including 11,12,13 and 0. Add one chooser tie case and one invalid-data case to rerun.

Use exact integer duration validation0..60 before the predicate. The independent12-minute task is separate from the worked10- and15-minute practices. Supply actual tie and invalid-data records plus expected reports; a label such as tie test is not a reproducible case. Preserve whether you wrote a prediction, made a paper trace or actually ran code.

Your first attempt and help record

My independent explanation, trace and limits

Attempt this case before feedback. Record any help. Earlier examples below are further practice; a case already practised with feedback cannot establish fresh transfer.

Your teacher has the independent answer and scoring criteria. Complete the matching workbook week to keep your practice **evidence** together.

Use feedback afterwards

Attempt the independent case before hints. Record whether you worked alone, used a hint or followed a model. After feedback, fix the first unsupported step and explain why it changed.

A later check uses a changed case, not a copied answer.

▼ Earlier wording, alternatives and picture transcripts

Independent case: Repair independently

Week 35 Independent application

Close the worked explanation. Apply the idea to this different question without copying.

After changing a scoring weight, the system fixes one request but breaks two previously correct ones. What should the improvement record say?

My answer, with a trace, calculation or supporting **evidence**

> **Write or draw your reasoning and evidence.**

Create and test

Change one relevant detail. Predict the result before testing. Include a boundary or missing-information case when it helps reveal a limit.

My changed **input** and expected result

> **Write or draw your reasoning and evidence.**

Connect to your project

Publish the cause, one change, fixed test set, **before/after evidence**, and remaining limitation.

The **evidence** I will keep

Write or draw your reasoning and evidence.

Transfer practice from the original programme

After a scoring change, four old failures pass but two previous passes fail. On the original ten cases there were six passes. Compute the new total and explain what else should be reported.

Explain the mechanism, evidence and one limit.

Use feedback to improve

MAIN LESSON · SECTION 11

Preserve the repair and its remaining limits

> A higher aggregate can conceal a newly introduced failure.

This week’s concrete contribution

For each recommender revision keep the versioned requirement, original failed input, expected and actual output, precise change, targeted check, earlier passing cases rerun and remaining failures. If reused final cases guide a change, relabel them development/regression evidence and obtain fresh final cases. Report gains and losses by case, not only the new total.

Keep publication decisions with the project owner; this lesson creates a local evidence record.

For each recommender revision, retain cause, code change, targeted test, **regression** results and unresolved issues.

Keep the checkpoint and human choice

THIS WEEK'S CHECKPOINT Publish the cause, one change, fixed test set, **before/after evidence**, and remaining limitation.

SYSTEM MAP Reader need → allowed **input**s → transparent rule or score → ranked candidates → explanation → test record → **human decision**

What will you add, change, or verify in the project this week?

> **Write or draw your reasoning and evidence.**

What should remain under **human control**?

> **Write or draw your reasoning and evidence.**

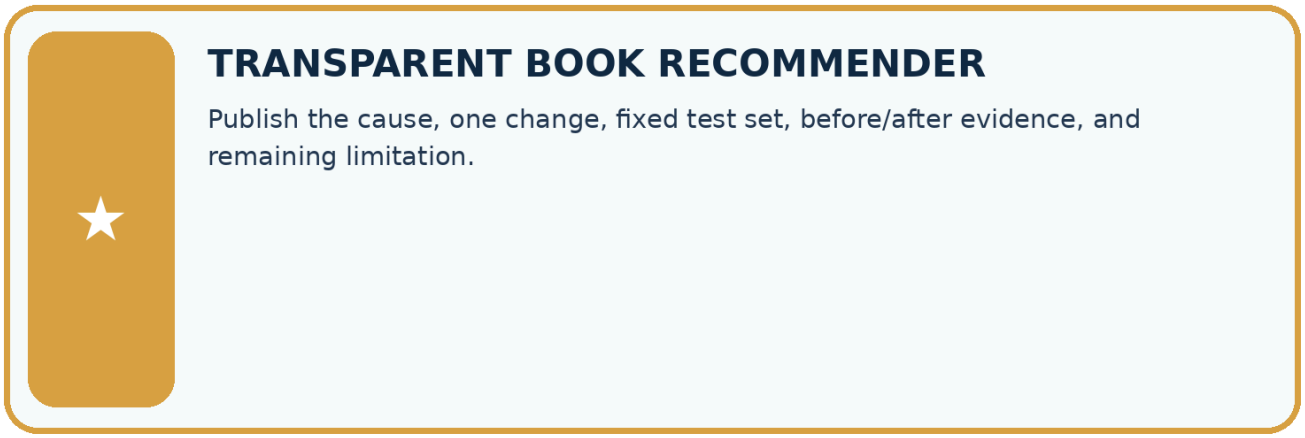
RESPONSIBLE DESIGN Collect only necessary information. Show uncertainty and limits. Never confuse a ranking with a fact about a person.

▼ Earlier wording, alternatives and picture transcripts

CUMULATIVE PROJECT

Transparent Book Recommender

Every week adds one inspectable piece. The system must show how it reached a result.



Evidence for your project

MAIN LESSON · SECTION 16

Finish: explain what changed and which checks passed

> Retain the original failure and limits on the repair claim.

Explain without notes

- Close your notes. Explain this goal in your own words: Repair a specific observed failure

My deciding step, evidence and one remaining question

Plan a changed-case return

If you are unsure, return to the worked picture and explain one arrow. If you are ready, change an **input** or assumption and justify your **prediction**.

Save your workbook **evidence**. At the next review, try a different case before opening your old answer.

Reference bank and extra practice

Choose the reference that helps your next step.

▼ **Reference · Words for the main lesson and extension**

REFERENCE · SECTION 5

Reference · Words for the main lesson and extension

> Use each word with a concrete record or calculation.

Core words

Word	My own example	Not this / boundary
controlled improvement	A revision whose effect is checked under comparable conditions rather than assumed from a changed result.	
before/after	Results recorded prior to and following a change; other changes can affect the comparison.	
tradeoff	A situation where improving one desired property can make another harder to achieve.	
regression	In testing, a new failure introduced by a change; in modelling, prediction of a numeric target. Use the meaning indicated by the lesson.	

Main explanation terms

Word	Meaning
Regression suite	Earlier cases rerun after a change

Word	Meaning
Change log	A record of what changed and why
Root cause	The underlying reason for a failure
Residual risk	A limitation or possible failure remaining after a repair

Extension terms

Term	Meaning
Unit test	A check of one small component
Integration test	A check of connected components
Fixture	A controlled input used in a test
Oracle	An independent source of the expected result

▼ Earlier wording, alternatives and picture transcripts

VOCABULARY LAB

Words You Can Use to Reason

A definition matters when you can recognize the idea, use it, and contrast it with a near-miss.

1

CONTROLLED IMPRC

one planned change tied to evidence

2

BEFORE/AFTER

results from the same test before and after





Words used in the closer explanation

Terms for Testing functions and interfaces

▼ **Extra challenge · Choose a stronger explanation**

REFERENCE · SECTION 10

Extra challenge · Choose a stronger explanation

> Choose the next useful challenge rather than repeating the ladder.

Thinking ladder

- 1 • NAME: Define the concept without repeating the textbook.
- 2 • TRACE: Show the information, state, branch, calculation, or **evidence** path.
- 3 • APPLY: Solve a different example independently.
- 4 • CHALLENGE: Find a boundary case, counterexample, or unfair outcome.
- 5 • EXPLAIN LIMITS: Say when the result should not be trusted or used alone.
- My strongest **evidence** of independent understanding:

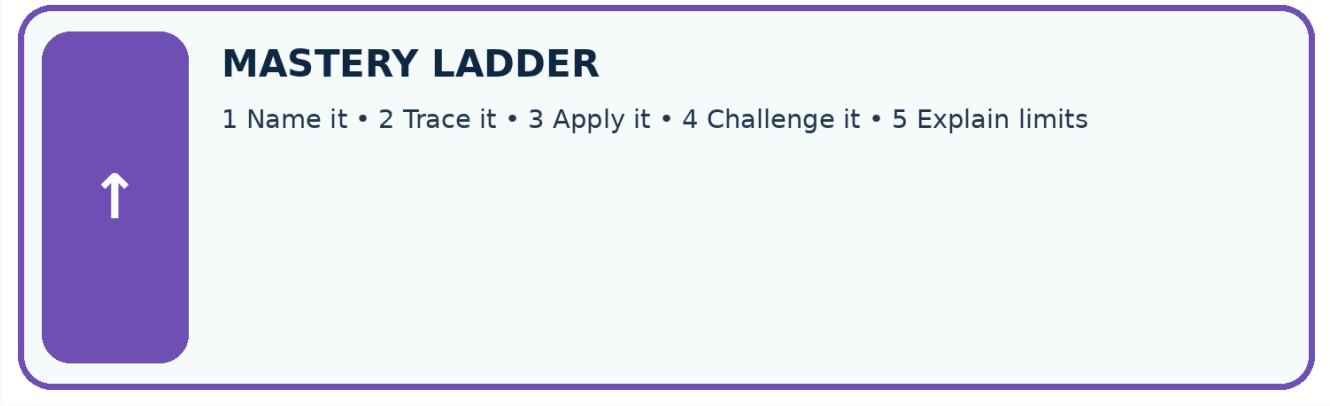
Write or draw your reasoning and evidence.

▼ Earlier wording, alternatives and picture transcripts

MASTERY

From Knowing the Word to Owning the Idea

Move upward only when you can produce **evidence** without copying the worked example.



▼ Reference · Check your explanation

REFERENCE · SECTION 12

Reference · Check your explanation

> Use after practice; Repair a new boundary independently is the selected independent task.

Evidence and limits

- 1. My fresh example, trace, calculation, or design:
- 2. My **evidence** that it works (include a boundary or missing-data case):
- 3. One limitation, fairness concern, or situation needing human review:

Write or draw your reasoning and evidence.

Write or draw your reasoning and evidence.

Write or draw your reasoning and evidence.

Self-check

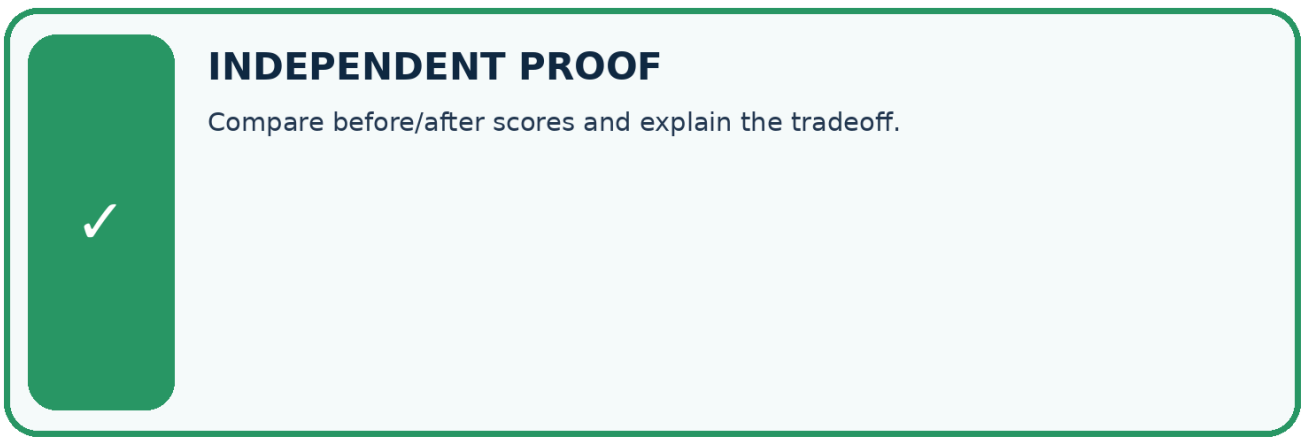
SELF-CHECK ☐ I explained the **mechanism** ☐ I used **evidence** ☐ I tested a fresh case ☐ I named a limitation ☐ I can answer 'why?'

▼ Earlier wording, alternatives and picture transcripts

INDEPENDENT MASTERY PROOF

Checklist for this week's explanation

Use after practice. Repair a new boundary independently is the selected independent task.



Require a validated predicate, explicit expected boundary outcomes and concrete tie/invalid fixtures.

▼ **Extra practice · The earlier complete case**

REFERENCE · SECTION 13

Extra practice · The earlier complete case

> Try the question before opening the worked explanation.

Question and given facts

A repair raises success from 7/10 to 9/10 on the same validation set.

What belongs in the improvement claim?

Your explanation

- Which detail supports the answer, and why does it matter?

> **Write or draw your reasoning and evidence.**

▼ **Check after trying**

Confirm fixed criteria, inspect remaining and introduced failures, then test the chosen version on fresh final cases.

Evidence or stage	Value or result
Before	Save results and failures
Revision	Change one identified cause
Retest	Use the same development cases
Final check	Reserve fresh cases

▼ Earlier wording, alternatives and picture transcripts

Week 35 Worked case

Use the supplied details to practise the weekly concept before attempting the independent question. This case extends the illustrated lesson.

The situation

The question

Worked explanation

Explain it in your own words

Trace the supplied case visually

Improve with comparable evidence

A repair raises success from 7/10 to 9/10 on the same validation set.

What belongs in the improvement claim?

Confirm fixed criteria, inspect remaining and introduced failures, then test the chosen version on fresh final cases.

▼ Reference · Earlier checklist

Reference · Earlier checklist

> Use when it helps inspect the evidence.

Checklist and first prediction

By the end, I can	How I will show it
Repair a specific observed failure	Explain the deciding step, show a trace or test, and state one limit.
Check that earlier behavior still works	Explain the deciding step, show a trace or test, and state one limit.
Keep a truthful change and failure record	Explain the deciding step, show a trace or test, and state one limit.

Remember → watch and explain → try with help → investigate → try independently → keep project **evidence**. Your teacher may spread this lesson across several classes.

Keep your first **prediction**. Compare it with what the **evidence** shows and explain any change.

▼ Earlier wording, alternatives and picture transcripts

Start with this question: A fictional recommender excludes books at a page limit that is intended to include equality.

▼ Extra practice · Pictures, feedback and repair

REFERENCE · SECTION 17

Extra practice · Pictures, feedback and repair

> Try before opening feedback; keep the selected independent task separate.

First supported question

- A fictional recommender excludes books at a page limit that is intended to include equality.
- Which labelled step matches this detail? Rerun 99,100,101; invalid pages; tied scores; reversed row order.
- 1. Identify cause
- 2. Expected revised result
- 3. Challenge repair
- 4. Focused repair
- Hint if needed: Read the steps in the pictorial case. Match the supplied detail to the stage that performs or records it.

Practice 1: explain the deciding evidence

> **My selected conclusion, deciding fact and explanation**
>
> ---
>
> ---
>
> ---
>
> ---

▼ **Check after trying**

After your attempt, compare with the supported explanation: Challenge repair Rerun 99,100,101; invalid pages; tied scores; reversed row order.

A focused change makes the claim about what caused the error testable: the equality comparison rule caused this specific failure. **Regression** tests check surrounding behavior that should remain unchanged.

A tie case and reversed **input** order are necessary because a correct boundary does not ensure deterministic selection. Log the old failure and revised replay separately.

Changed-case question

- Changed-case practice: An alternative patch uses pages <= 101 instead of the required pages <= 100. Which conclusion is supported by the supplied facts?
- 1. We can decide the result without examining the changed **input** or the procedure.
- 2. This one example guarantees correct results for every future **input**.
- 3. C is wrongly admitted as well as A.
- Hint if needed: Use the changed condition and the deciding step in the pictorial trace. The answer must say what follows from those facts.

Practice 2: explain the deciding evidence

My selected conclusion, deciding fact and explanation

▼ **Check changed case**

After your attempt, compare with the supported explanation: C is wrongly admitted as well as A. Changing the maximum alters the contract instead of repairing its comparison.

Repair and keep evidence

Review Test Diverse Users (ai-2/34). Goal: Evaluate across varied cases. Short practice: Can these records establish performance for different user groups?. Return to this week and explain what you changed.

Review Project Problem and Success Test (ai-2/32). Goal: Define a small application purpose. Short practice: Write an equality test and explain which requirement it checks.. Return to this week and explain what you changed.

Review **Evidence** and Sources (ai-2/29). Goal: Explain the **mechanism** and terminology of claim support and citation evaluation.

Short practice: Why is the original answer not fully supported despite having citations on every claim?. Return to this week and explain what you changed.

The browser lab lets you edit the supported Python examples already included in this lesson. Predict first, run, inspect the actual **output** or error, change one **input**, then run again.

Keep code, **prediction**s, actual results and a reasoned revision together. On paper, label your result as a trace rather than an execution.

Use Download completed work in the hosted lesson to export your answers, reflections, practice feedback and code-run **evidence**. Open the HTML copy to read or print it.

Drafts stay in this browser when storage is available; clear drafts on a shared device.

▼ Earlier wording, alternatives and picture transcripts

Use this supported practice once, in the browser or on paper. Keep the reserved independent assessment for a separate first attempt before teacher feedback.

Repair a missing building block

Predict, run and debug

Keep your completed work

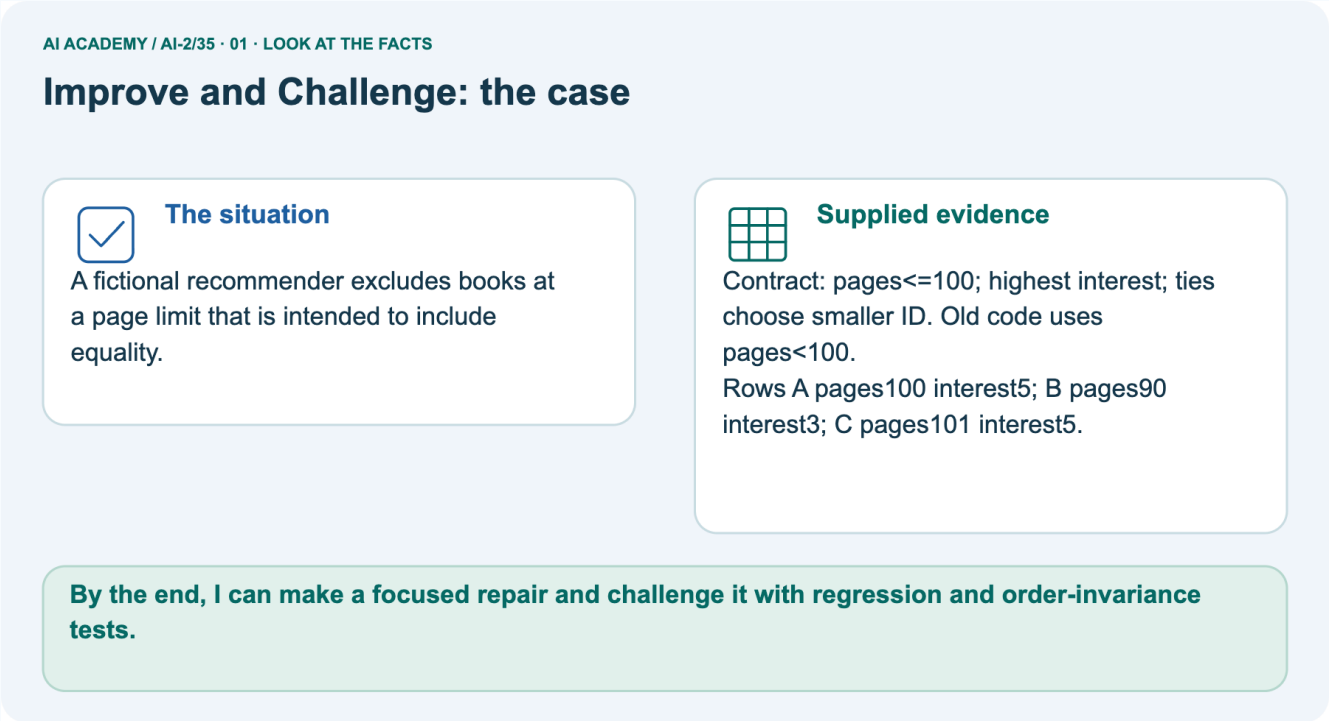
▼ Reference · The six-picture journey

REFERENCE · SECTION 18

Reference · The six-picture journey

> Revisit the picture that serves the current question.

01. Improve and Challenge: the case



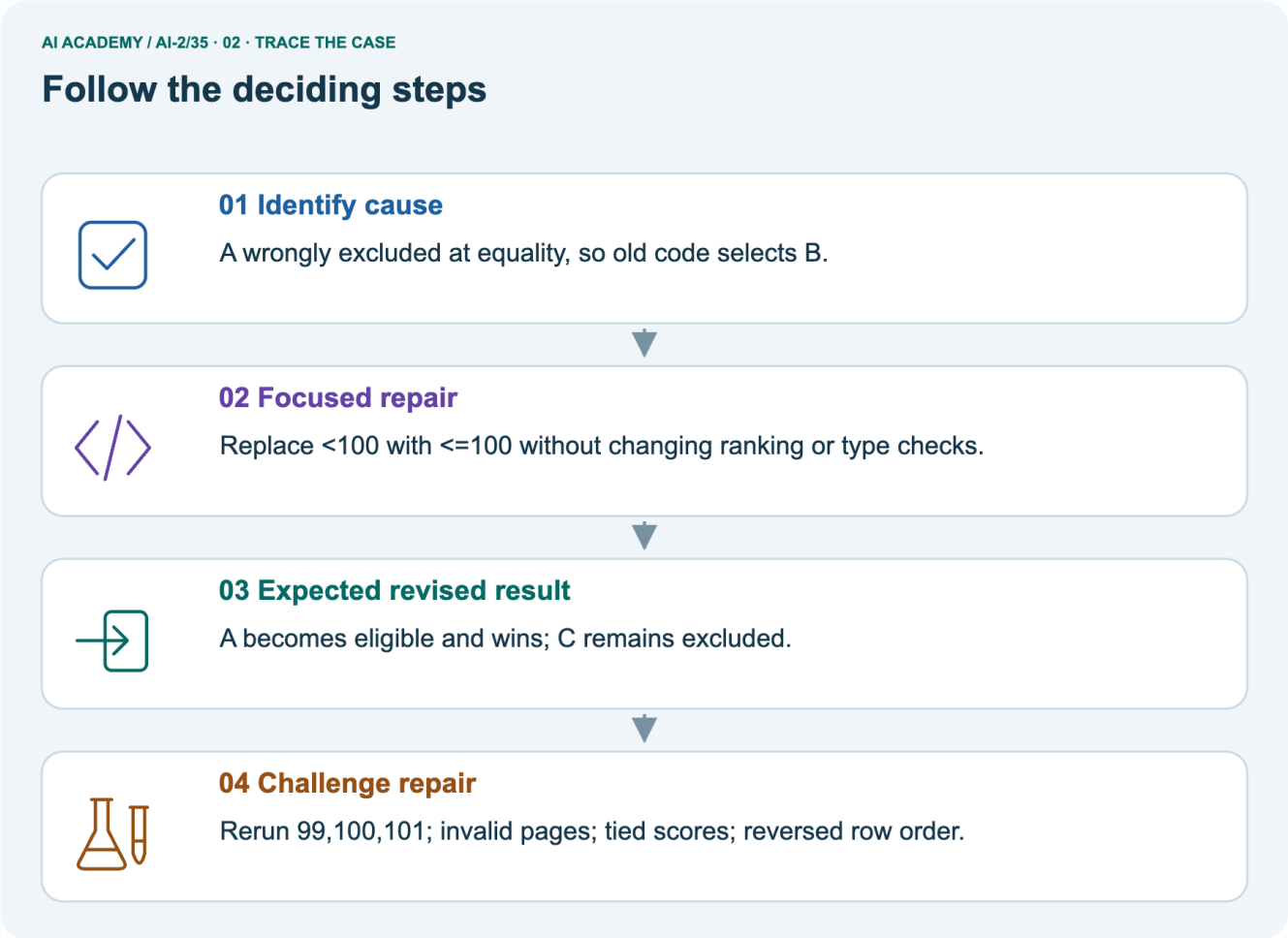
Follow the labelled picture with the matching explanation. Open the vector diagram

A fictional recommender excludes books at a page limit that is intended to include equality.
Supplied facts: Contract: pages<=100; highest interest; ties choose smaller ID. Old code uses pages<100.

Rows A pages100 interest5; B pages90 interest3; C pages101 interest5.

Point to the given fact that matters most. Explain why it matters.

02. Follow the deciding steps

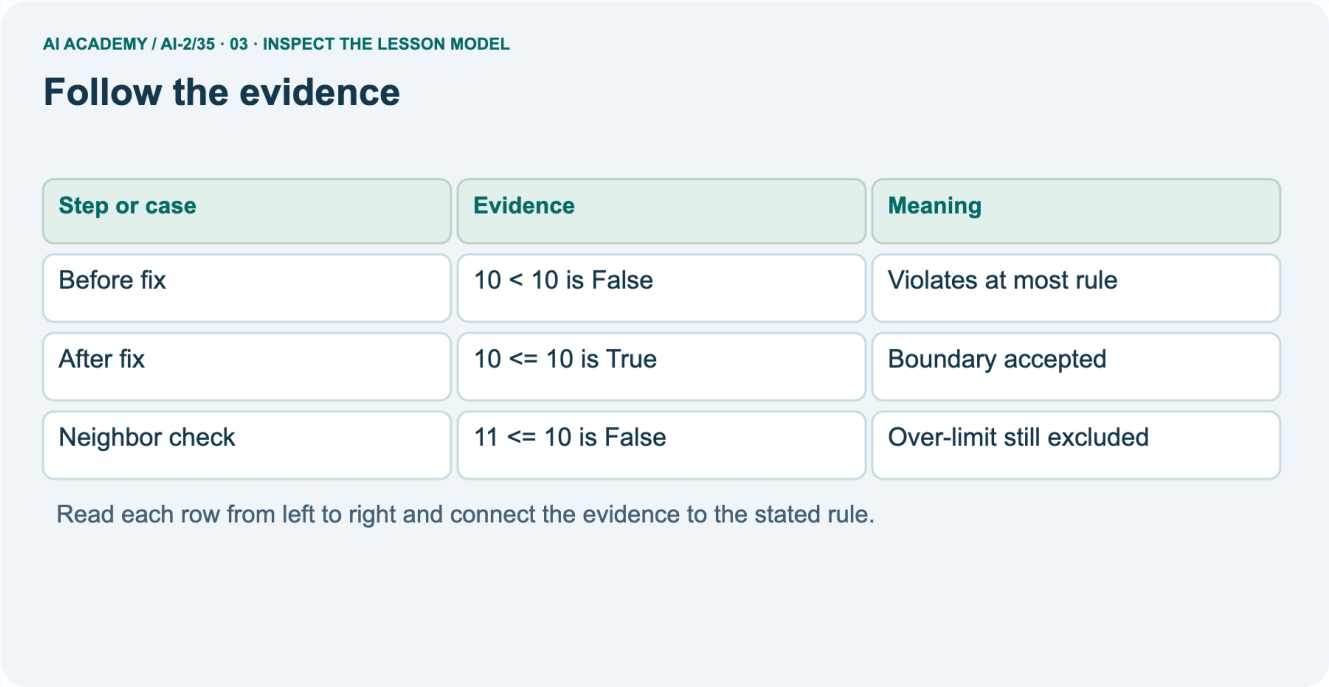


Follow the labelled picture with the matching explanation. Open the vector diagram

Identify cause: A wrongly excluded at equality, so old code selects B. → Focused repair: Replace <100 with <=100 without changing ranking or type checks. → Expected revised result: A becomes eligible and wins; C remains excluded. → Challenge repair: Rerun 99,100,101; invalid pages; tied scores; reversed row order.

Point to each numbered stage and explain which recorded **input** it uses.

03. Follow the evidence

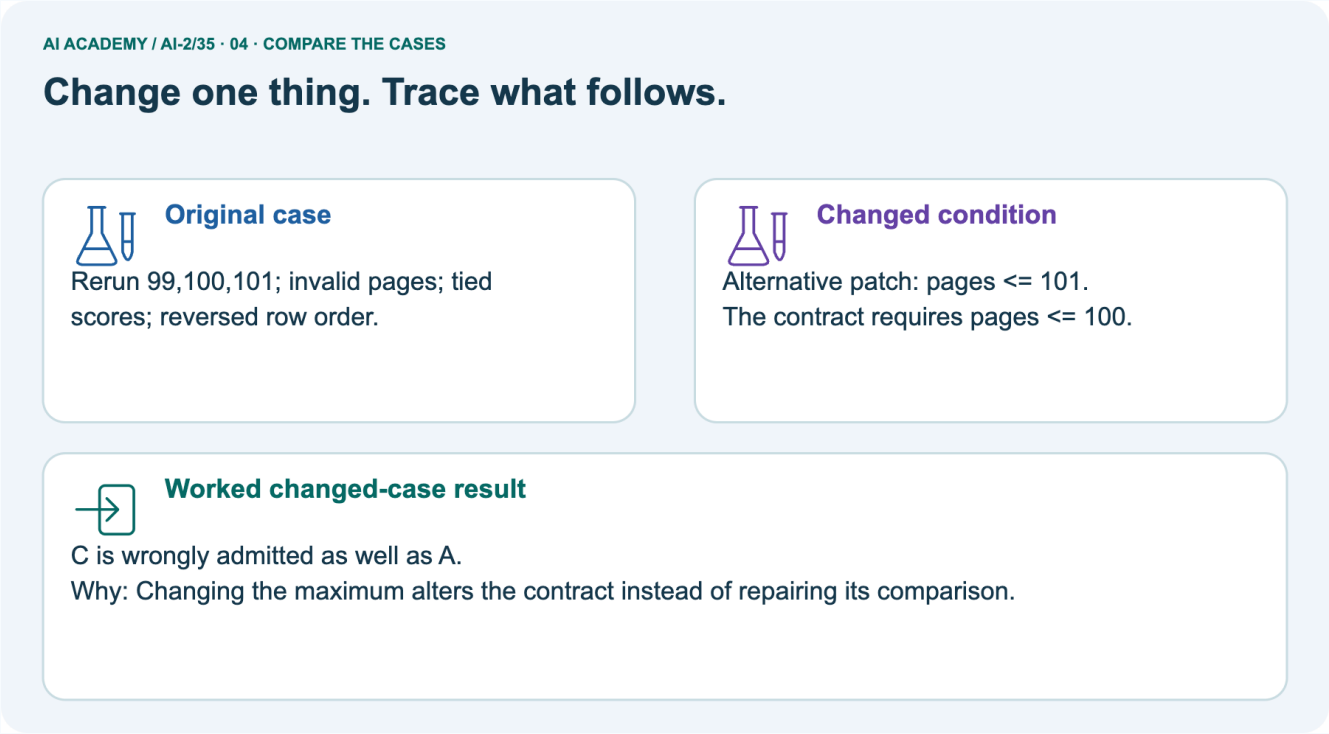


Follow the labelled picture with the matching explanation. Open the vector diagram

Follow the **evidence**. Before fix | 10 < 10 is False | Violates at most rule. After fix | 10 <= 10 is True | Boundary accepted. Neighbor check | 11 <= 10 is False | Over-limit still excluded

Read the labels and values. Explain the relationship the model shows.

04. Change one thing. Trace what follows.

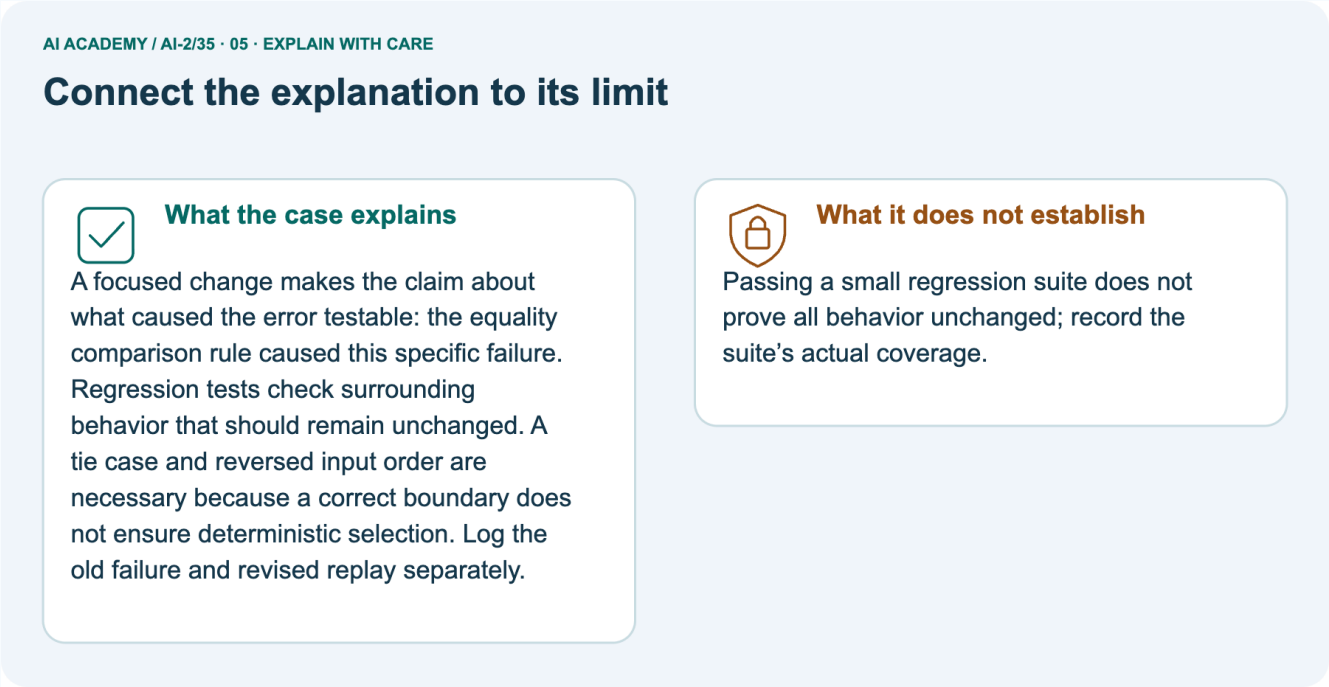


Follow the labelled picture with the matching explanation. Open the vector diagram

Original case: Rerun 99,100,101; invalid pages; tied scores; reversed row order. Changed condition: An alternative patch uses pages <= 101 instead of the required pages <= 100. Expected result: C is wrongly admitted as well as A. Reason: Changing the maximum alters the contract instead of repairing its comparison.

Predict before checking the worked changed-case result. Name what changed and what the supplied facts allow you to conclude.

05. Connect the explanation to its limit



Follow the labelled picture with the matching explanation. Open the vector diagram

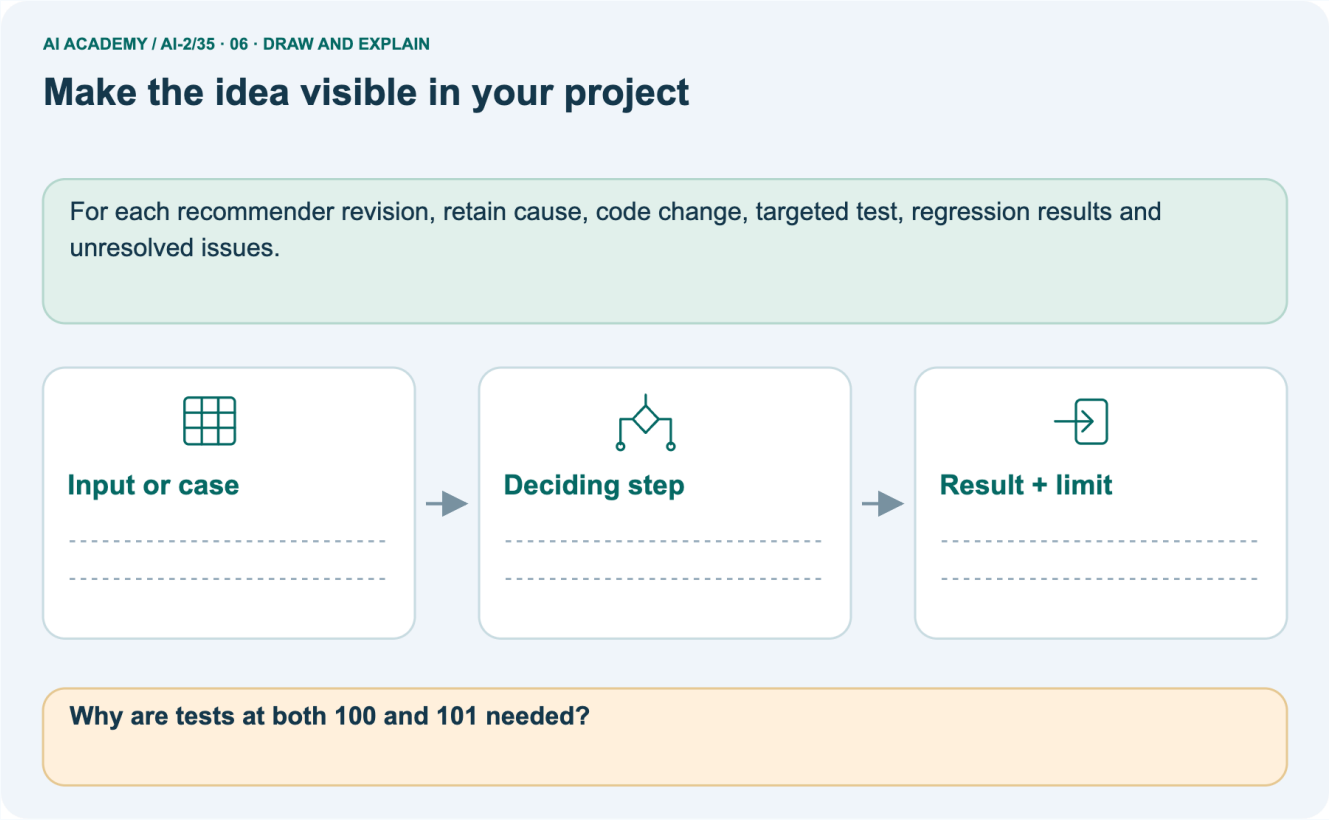
A focused change makes the claim about what caused the error testable: the equality comparison rule caused this specific failure. **Regression** tests check surrounding behavior that should remain unchanged.

A tie case and reversed **input** order are necessary because a correct boundary does not ensure deterministic selection. Log the old failure and revised replay separately.

Limit: Passing a small **regression suite** does not prove all behavior unchanged; record the suite's actual coverage.

Explain the case, then point to a limit on the claim.

06. Make the idea visible in your project



Follow the labelled picture with the matching explanation. Open the vector diagram

For each recommender revision, retain cause, code change, targeted test, **regression** results and unresolved issues. Draw your **input**, deciding step, result and limit. Supported question: Why are tests at both 100 and 101 needed?

Draw your project trace and justify the deciding step using **evidence**.

▼ Earlier wording, alternatives and picture transcripts

01. Improve and Challenge: the case

02. Follow the deciding steps

03. Follow the evidence

04. Change one thing. Trace what follows.

05. Connect the explanation to its limit

06. Make the idea visible in your project

▼ Optional interactive practice and completed work

Save or print your responses for your portfolio. Progress and drafts on the hosted website stay in the current browser. Use only fictional or teacher-approved data.